TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

CONSTRAINT SATISFACTION PROBLEMS

Crypto Arithmetic & Map Colouring

Lab 5 Report

Submitted By:

Name: **Chandan Kumar Shah**

Roll No: **080BCT023**

Date: **2083-04-02**

Submitted To:

Department of Electronics
and Computer Engineering
Institute of Engineering,
Pulchowk Campus

Subject: Artificial Intelligence

Contents

1 Objectives

The primary objectives of this laboratory exercise are:

1. To understand the concept of **Constraint Satisfaction Problems (CSPs)** and how constraints limit the values variables may take.
2. To implement and solve the **Crypto Arithmetic Problem** (`SEND + MORE = MONEY`) using Prolog's constraint logic programming facilities.
3. To formulate and solve the **Australia Map Colouring Problem** as a CSP using Prolog.
4. To implement the same map colouring problem in Python using brute-force search.
5. To compare declarative and imperative approaches to solving CSPs.

2 Introduction

A **Constraint Satisfaction Problem (CSP)** is defined by a set of variables, a domain of possible values for each variable, and a set of constraints that restrict the combinations of values the variables may take simultaneously. Formally, a CSP is a triple (X, D, C) where:

- $X = \{x_1, x_2, \ldots, x_n\}$ is the set of variables,
- $D = \{D_1, D_2, \ldots, D_n\}$ is the set of domains, with D_i the possible values of x_i,
- C is the set of constraints, each specifying allowed combinations for a subset of variables.

A *solution* is an assignment of each variable to a value from its domain such that every constraint is satisfied. Many AI problems—scheduling, configuration, puzzle solving, and planning—can be naturally expressed as CSPs.

Constraint Logic Programming (CLP) extends logic programming with constraint solving. In SWI-Prolog, the `library(clpfd)` module provides finite-domain constraints such as `ins/2`, `all_different/1`, and arithmetic constraints over integers, making it convenient to model CSPs declaratively.

3 Theory

3.1 Crypto Arithmetic Problem

Crypto arithmetic is a classic CSP in which each letter stands for a distinct digit. The well-known puzzle `SEND + MORE = MONEY` is written column-wise (right to left) with carries $C_1, C_2, C_3, C_4 \in \{0, 1\}$:

$$\begin{array}{cccccc} & S & E & N & D \\ + & M & O & R & E \\ \hline M & O & N & E & Y \end{array}$$

The column equations are:

$$D + E = Y + 10C_1 \tag{1}$$
$$N + R + C_1 = E + 10C_2 \tag{2}$$
$$E + O + C_2 = N + 10C_3 \tag{3}$$
$$S + M + C_3 = O + 10C_4 \tag{4}$$
$$M = C_4 \tag{5}$$

Additional constraints are that all letters represent different digits and that leading digits S and M are non-zero.

3.2 Map Colouring Problem

The map-colouring problem asks whether a map can be coloured with a given number of colours so that no two adjacent regions share the same colour. The Four Colour Theorem guarantees that any planar map can be coloured with at most four colours; our Australia map instance uses three colours.

The Australia map can be modelled with seven variables (one per region) with domain {red, green, blue} and a *different-colour* constraint for every pair of adjacent regions:

- WA borders NT and SA.
- NT borders WA, Q, and SA.
- Q borders NT, NSW, and SA.
- NSW borders Q, V, and SA.
- V borders NSW and SA.
- SA borders WA, NT, Q, NSW, and V.
- Tasmania (T) is an island and has no mainland neighbours.

4 Methodology

The following tools and environment were used:

- Operating System: macOS
- Prolog interpreter: SWI-Prolog 10.0.2
- Java SDK: OpenJDK 17
- Editor / IDE: Any text editor with Prolog and Java syntax support

For each task, the problem was first modelled by identifying variables, domains, and constraints. The Prolog implementation uses CLPFD constraints and backtracking search via `label/1`. The Python implementation uses a brute-force search over the Cartesian product of colours.

5 Implementation

5.1 Task 1: Crypto Arithmetic Problem

File: `1.pl`

```
% SEND + MORE = MONEY cryptarithmetic puzzle
% Using clpfd (Constraint Logic Programming over Finite Domains)

:- use_module(library(clpfd)).

solution(S, E, N, D, M, O, R, Y) :-
    Vars = [S, E, N, D, M, O, R, Y],
    Vars ins 0..9,
    % All letters must have distinct values
    all_different(Vars),
    % Leading digits cannot be 0
    S #\= 0,
    M #\= 0,
    % C4 must be 1 (carry from the leftmost column)
    M #= 1,
    % Column arithmetic (right to left)
    D + E #= Y + 10 * C1,
    N + R + C1 #= E + 10 * C2,
    E + O + C2 #= N + 10 * C3,
    S + M + C3 #= O + 10 * C4,
    C4 #= 1,
    % Label (find the solution)
    label(Vars).
```

Listing 1: Prolog program for SEND + MORE = MONEY

Execution command:

```
$ swipl -q -t "solution(S,E,N,D,M,O,R,Y), writeln([S,E,N,D,M,O,R
    ,Y])." 1.pl
```

Output:

```
[9,5,6,7,1,0,8,2]
```

This corresponds to $9567 + 1085 = 10652$.

5.2 Task 2: Map Colouring in Prolog

File: `map_coloring.pl`

```
% Australia Map Colouring Problem (Constraint Satisfaction)
% Regions: WA, NT, Q, NSW, V, SA, T
% Domain : {red, green, blue}  (encoded as 1, 2, 3)
% Constraint: adjacent regions must have different colours.

:- use_module(library(clpfd)).

% solve(-Colouring)
% Colouring is a list Region-Colour where Colour is in 1..3.
solution(Colouring) :-
    Colouring = [
        wa-WA, nt-NT, q-Q, nsw-NSW, v-V, sa-SA, t-T
    ],
    Colours = [WA, NT, Q, NSW, V, SA, T],
    Colours ins 1..3,

    % Adjacency constraints: neighbouring regions must differ.
    WA #\= NT, WA #\= SA,
    NT #\= Q,  NT #\= SA,
    Q  #\= NSW, Q  #\= SA,
    NSW #\= V, NSW #\= SA,
    V #\= SA,

    label(Colours).

% colour_name(+Code, -Name)
colour_name(1, red).
colour_name(2, green).
colour_name(3, blue).

% print_solution(+Colouring)
print_solution(Colouring) :-
    write('Australia Map Colouring Solution:'), nl,
    forall(
        member(Region-Code, Colouring),
        ( colour_name(Code, Name),
          format('  ~w = ~w~n', [Region, Name]) )
    ).

% all_solutions(-Solutions)
all_solutions(Solutions) :-
    findall(Colouring, solution(Colouring), Solutions).
```

Listing 2: Prolog program for Australia map colouring

Execution command:

```
$ swipl -q map_coloring.pl
```

Output:

```
Australia Map Colouring Solution:
  wa = red
  nt = green
  q = red
  nsw = green
  v = red
  sa = blue
  t = red
```

5.3 Task 3: Map Colouring in Python

File: map_coloring.py

```python
# Australia Map Colouring Problem (Constraint Satisfaction)
# Regions: WA, NT, Q, NSW, V, SA, T
# Domain : {red, green, blue}
# Constraint: adjacent regions must have different colours.

from itertools import product

REGIONS = ["WA", "NT", "Q", "NSW", "V", "SA", "T"]
COLOURS = ["red", "green", "blue"]

# Adjacency list (undirected).
ADJACENT = {
    "WA": ["NT", "SA"],
    "NT": ["WA", "Q", "SA"],
    "Q": ["NT", "NSW", "SA"],
    "NSW": ["Q", "V", "SA"],
    "V": ["NSW", "SA"],
    "SA": ["WA", "NT", "Q", "NSW", "V"],
    "T": [],  # Tasmania has no mainland neighbours.
}


def is_valid(assignment):
    """Check whether every adjacency constraint is satisfied."""
    for region, colour in assignment.items():
        for neighbour in ADJACENT[region]:
            if assignment.get(neighbour) == colour:
                return False
    return True


def solve():
    """Find the first valid colouring by brute-force search."""
    for combo in product(COLOURS, repeat=len(REGIONS)):
        assignment = dict(zip(REGIONS, combo))
        if is_valid(assignment):
```

```
            return assignment
    return None


if __name__ == "__main__":
    solution = solve()
    if solution:
        print("Australia Map Colouring Solution:")
        for region in REGIONS:
            print(f"  {region} = {solution[region]}")
    else:
        print("No valid colouring found.")
```

Listing 3: Python program for Australia map colouring

Execution command:

```
$ python3 map_coloring.py
```

Output:

```
Australia Map Colouring Solution:
  WA = red
  NT = green
  Q = red
  NSW = green
  V = red
  SA = blue
  T = red
```

6 Results

Table 1: Summary of results for Lab 5

Task	Problem	Result
1	Crypto Arithmetic (Prolog)	$S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2$
2	Map Colouring (Prolog)	WA/NSW/T red; NT/V green; SA blue
3	Map Colouring (Python)	Same valid 3-colouring as Prolog

The Prolog and Python implementations both produced the same valid colouring. This confirms that the CSP formulation is correct and that the same constraints can be solved by different programming paradigms.

7 Discussion

The crypto arithmetic program demonstrates how arithmetic constraints can be combined with all-different and non-zero constraints to solve a puzzle declaratively. Using

`library(clpfd)` significantly reduces the amount of code compared to a generate-and-test approach.

For the map-colouring problem, the Prolog solution is concise because constraints are stated directly. The Java solution requires explicit backtracking and adjacency checking, while the Python version uses a straightforward brute-force search over the Cartesian product of colours. All three produce the same result, which shows that once the CSP is well defined it can be implemented in many ways.

One observation is that Tasmania has no adjacent regions in the given map, so it can take any of the three colours. This reflects the real-world geography of Tasmania as an island state.

8 Conclusion

This laboratory exercise successfully demonstrated the modelling and solution of constraint satisfaction problems using both logic programming and imperative programming. The crypto arithmetic and map-colouring problems were correctly solved, and the equivalence of the Prolog and Java results validates the CSP formulation.